

Drone-Remote Weapon Station Software - Architecture and Safety Classification (MIL-STD-882E)

1. Purpose

- This document explains the software architecture required for the Remote Weapon Station (RWS) and identifies which software elements are safety-critical versus non-safety software in accordance with MIL-STD-882E principles.

2. Software Classification per MIL-STD-882E

- MIL-STD-882E requires identification of software safety criticality and independence between safety authority and functional software.

3. Safety-Critical Software

- Independent Safety Monitor firmware – SAFE authority, watchdog, torque disable, fire inhibit
- Fire interlock logic – dual-channel firing authorization
- Drive enables gating logic – hardware/software interface ensuring torque-off on fault

4. Non-Safety (Functional) Software

- Operator GUI and HMI
- Video processing and streaming
- Aiming and stabilization algorithms
- Ballistic computation
- Tracking assist functions
- Data logging and telemetry display

5. Software Architecture Overview

- The RWS software is partitioned into Operator GUI, Fire Control Computer software, Independent Safety Monitor firmware, and Fire Interlock logic to ensure independence and fail-safe behavior.

6. Independence Principle

- Safety Monitor firmware operates independently of the Fire Control Computer to enforce SAFE state during comm loss, power fault, or software malfunction.

7. Safety Mechanisms

- Heartbeat supervision
- Dual-channel fire enables
- Hard torque disables
- Limit switch enforcement
- Watchdog timers

8. Verification Approach

- Verification includes fault injections, watchdog timeout testing, comm loss testing, and fire inhibit validation.

Software Safety Upgrade (MIL-STD-882E Extended)**9. Software Integrity Allocation**

Software Element	Integrity Level
ISM Firmware	High
Fire Interlock Logic	High
FCC Control Software	Medium
Operator GUI	Low

10. Software Safety Requirements Traceability

- Hazard → Safety Requirement → Software Implementation → Verification mapping established.

11. Software Independence Argument

- ISM firmware operates independently of FCC ensuring SAFE state despite FCC failure.

12. Software Safety Case Section

- Top claim: Software cannot produce hazardous firing without safety authorization.

13. Software Hazard Mapping

- Runaway motion → torque disable logic; unintended fire → interlock gating; comm loss → watchdog SAFE.

14. JSSEH Alignment

- Software lifecycle includes hazard analysis, traceability, verification, configuration control.

15. Cybersecurity Interface (RMF)

- Authentication, encrypted comm transport, anti-spoofing checks integrated.

16. Software Verification Matrix

- Includes HIL testing, fault injection, watchdog timeout validation, GUI usability validation.

17. Safety Software Package

- Software Architecture Diagram: FCC modules (comms, control, video, logging), ISM firmware safety core, interlock logic.
- FCC Software Decomposition: Comms service, turret control service, fire control service, health monitoring, logging.
- ISM Firmware State Machine: SAFE → ARMED → FIRING with fault transition to SAFE.
- GUI Safety Screen Specification: Clear SAFE/ARMED annunciation, dual confirmation firing, alarm prioritization.
- Message Protocol Tables: GCS↔FCC Ethernet messages; FCC↔ISM heartbeat; CAN motion and fault frames.
- Software FMEA: FCC crash mitigated by ISM SAFE; GUI error mitigated by interlock; comm corruption mitigated by CRC.
- Software Hazard Log Extension: Hazards mapped to software mitigations and verification evidence.
- Software Verification Procedures: HIL tests, fault injection, watchdog validation, endurance tests.
- Software SAR Evidence Section: Logs, fault reports, verification artifacts supporting safety approval.

UPGRADED DUAL-CHANNEL + ISM SAFETY ARCHITECTURE (INTEGRATED)

1. Safety Authority Architecture Upgrade

- The system is updated to include an Independent Safety Monitor (ISM) acting as the final safety authority.
- All actuator commands are validated and gated by the ISM prior to execution.

2. Safety Authority Separation (Critical Requirement)

- The ISM operates independently from the Fire Control Computer (FCC) and enforces a SAFE state regardless of FCC behavior, communication loss, or software malfunction.

3. Updated Software Architecture (Control vs Safety Separation)

- Control Computers generate motion commands, while ISM enforces safety constraints including torque disable, command validation, watchdog monitoring, and limit enforcement.

4. Hazard Mitigation Mapping

- Runaway motion → ISM torque disable
- Invalid command → ISM command validation
- Communication loss → ISM watchdog SAFE
- Timing fault → ISM supervision

5. FTA Contribution

- Top Event: Uncontrolled Motion
- Causes include ISM failure OR ISM pass-through of unsafe command.
- ISM is treated as the primary safety barrier.

6. Expanded FMECA – ISM Critical Failure Modes

- Failure Mode: ISM crash → Effect: Loss of safety authority → Severity: Catastrophic → Mitigation: Hardware fail-safe default OFF
- Failure Mode: ISM logic error → Effect: Unsafe command allowed → Mitigation: Minimal verified code
- Failure Mode: ISM comm loss → Effect: No supervision → Mitigation: Default SAFE
- Failure Mode: Watchdog failure → Effect: Fault undetected → Mitigation: Independent watchdog

7. STPA Control Structure

- Operator → FCC → ISM → Actuators
- Unsafe Control Actions include ISM allowing unsafe torque or failing to enforce SAFE.

8. Software Safety Classification

- ISM Firmware: High Integrity
- Interlock Logic: High
- FCC: Medium
- GUI: Low

9. Verification Enhancements

- Added ISM-specific tests:
 - Fault injection (invalid command)
 - Watchdog timeout
 - Communication loss
 - Corrupted data validation
 - Timing violation scenarios

10. Safety Case

- The system cannot produce hazardous motion because all actuator commands are gated by an independent safety authority (ISM) capable of enforcing a SAFE state under all failure conditions.